МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Языки программирования

Шаблонные классы

Руководитель _____ А.В. Проскурин
подпись, дата инициалы, фамилия

Обучающийся БПИ24-02, 24151430 _____ В.Т. Гордов
номер группы, зачетной книжки подпись, дата инициалы, фамилия

Красноярск 2025 г.

ЦЕЛЬ РАБОТЫ

Получение практических навыков разработки и отладки программ, с использованием шаблонов функций и классов.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с общей постановкой задачи.

2. Ознакомится с вариантом задания – соответствует вашему номеру в списке группы

(при нехватке заданий вариант задания вычисляется как номер_в_списке_группы - количество_заданий).

3. Разработать классы согласно варианту задания.

4. Написать и отладить программу на подготовленных наборах тестовых данных.

5. Подготовить отчет по лабораторной работе. Отчет должен включать в себя:

• титульный лист;

• цель лабораторной работы;

• постановку задачи;

• схему наследования классов (UML диаграмма классов);

• текст программы с комментариями;

• демонстрацию работы программы (Снимки экрана при выполнении действий

программы с описанием).

• краткие ответы на контрольные вопросы;

• выводы по лабораторной работе.

6. Защитить лабораторную работу перед преподавателем.

ПОСТАНОВКА ЗАДАЧИ

Необходимо разработать программу, в которой будет реализован шаблонный класс, выполняющий обработку массива данных согласно варианту задания.

В зависимости от оценки, на которую вы претендуете, необходимо выполнить следующие задания (Для каждой следующей оценки нужно выполнить ВСЕ предыдущие задания, если обратное не указано явно):

Удовлетворительно

1. Определить шаблонный класс для обработки массива данных согласно заданию.
2. Реализовать конструкторы класса: - по умолчанию; - копирования (получающий параметр «ссылка на класс») Реализовать деструктор класса.
3. Написать методы ввода-вывода данных.
4. Реализовать метод, выполняющий одно из двух действий вашего варианта задания.

Хорошо

5. Реализовать метод, выполняющий второе действие вашего варианта задания.
6. Реализовать перегрузку операций >> и << с использованием методов для ввода и вывода значений полей.
7. В функции **main** заполнять объект шаблона с помощью текстового файла. Выводить данные также в файл.

Отлично

8. Написать метод **GenerateRandomValues**() для генерации случайных данных в полях.
9. Модификация элемента (выбирает пользователь) массива в шаблонном классе:
   - через консольный ввод;
   - при помощи метода **SetRandomValue**(), генерирующего случайное значение.
10. Для реализации сравнения и присваивания необходимо реализовать перегрузку соответствующих операторов.

Вариант №5.

Напишите программу, в которой создайте шаблонный класс **TMatrix**, реализующий динамический двумерный массив и функции-методы:

- подсчета элементов массива (с отображением на экран), которые являются положительными и меньше среднеарифметического значения всех элементов своей строки;
- расчёта суммы отрицательных элементов массива.

Протестируйте вашу программу для работы с массивами различных типов: **float**, класса **дробное число** (которое представлено двумя компонентами). Этот класс необходимо создать.

ХОД РАБОТЫ

Файл **TMatrix.h**:

```cpp
#ifndef TMATRIX_H
#define TMATRIX_H

#include <iostream>
#include <fstream>
#include <vector>
#include <random>
#include <limits>
#include <algorithm>
using namespace std;

/**
 * @brief Шаблонный класс, реализующий динамический двумерный массив
 * и предоставляющий методы для работы с ним.
 *
 * @tparam T Тип элементов матрицы:
 * float (вещественное число)
 * Fraction (дробь вида a/b)
 */
template<typename T>
class TMatrix {
private:
    // Представление матрицы как вектора векторов (двумерный массив)
    vector<vector<T>> data;
    // Количество строк матрицы
    int rows;
    // Количество столбцов матрицы
    int cols;

public:
    // Конструктор по умолчанию (пустая матрица 0x0)
    TMatrix();

    // Конструктор с размерами (r x c)
    TMatrix(int r, int c);

    // Конструктор копирования
    TMatrix(const TMatrix& other);

    // Деструктор
    ~TMatrix();

    // Оператор присваивания
    TMatrix& operator=(const TMatrix& other);

    // Оператор сравнения
    bool operator==(const TMatrix& other) const;

    // Методы ввода/вывода данных матрицы
    void input();
    void output() const;
```

```
49      // Методы ввода/вывода данных матрицы
50      void input();
51      void output() const;
52
53      // Перегрузки операторов ввода/вывода (обёртка соответствующих методов)
54      friend istream& operator>>(istream& is, TMatrix& m) {
55          m.input();
56          return is;
57      }
58
59      friend ostream& operator<<(ostream& os, const TMatrix& m) {
60          m.output();
61          return os;
62      }
63
64      // Подсчёт элементов > 0 и < среднего по строке
65      int countPositiveBelowRowMean() const;
66      // Подсчёт суммы отрицательных элементов
67      T sumOfNegativeElements() const;
68
69      // Заполнение матрицы случайными значениями
70      void GenerateRandomValues(T min_val, T max_val);
71
72      // Модификация элемента матрицы (ввод с консоли)
73      void modifyElement(int row, int col);
74
75      // Модификация элемента матрицы (случайное значение)
76      void SetRandomValue(int row, int col, T min_val, T max_val);
77
78      // Загрузка из файла
79      void loadFromFile(const string& filename);
80
81      // Сохранение в файл
82      void saveToFile(const string& filename) const;
83  };
84
85  // --- РЕАЛИЗАЦИЯ МЕТОДОВ ШАБЛОНА ---
86
87  template<typename T>
88  TMatrix<T>::TMatrix() : rows(0), cols(0) {}
89
90  /**
91   * @brief Конструктор с параметрами.
92   * Создаёт матрицу размером r x c, заполняя элементы значением по умолчанию T().
93   *
94   * @param r Количество строк.
95   * @param c Количество столбцов.
96   */
97  template<typename T>
98  TMatrix<T>::TMatrix(int r, int c) : rows(r), cols(c) {
99      data.resize(rows, vector<T>(cols));
100 }
101
102 /**
103  * @brief Конструктор копирования.
104  * Создаёт новую матрицу, являющуюся копией переданной.
105  *
106  * @param other Матрица, из которой производится копирование.
107  */
108 template<typename T>
```

```cpp
/**
 * @brief Конструктор копирования.
 * Создаёт новую матрицу, являющуюся копией переданной.
 *
 * @param other Матрица, из которой производится копирование.
 */
template<typename T>
TMatrix<T>::TMatrix(const TMatrix& other) : rows(other.rows), cols(other.cols) {
    data = other.data;
}

/**
 * @brief Деструктор.
 */
template<typename T>
TMatrix<T>::~TMatrix() {}

/**
 * @brief Оператор присваивания.
 * Присваивает значение полей одной матрицы другой.
 *
 * @param other Правый операнд присваивания.
 * @return Ссылку на текущий объект (*this).
 */
template<typename T>
TMatrix<T>& TMatrix<T>::operator=(const TMatrix& other) {
    if (this != &other) {
        rows = other.rows;
        cols = other.cols;
        data = other.data;
    }
    return *this;
}

/**
 * @brief Оператор сравнения.
 * Проверяет, равны ли две матрицы (по размеру и содержимому).
 *
 * @param other Матрица для сравнения.
 * @return true, если матрицы равны, иначе false.
 */
template<typename T>
bool TMatrix<T>::operator==(const TMatrix& other) const {
    return (rows == other.rows && cols == other.cols && data == other.data);
}

/**
 * @brief Метод ввода матрицы с консоли.
 * Сначала запрашивает количество строк и столбцов.
 * Затем поэлементно запрашивает значения.
 */
template<typename T>
void TMatrix<T>::input() {
    cout << "Enter number of rows: ";
    cin >> rows;
    cout << "Enter number of columns: ";
    cin >> cols;

    data.resize(rows, vector<T>(cols));

    for (int i = 0; i < rows; ++i) {
```

```
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << "Enter element [" << i << "][" << j << "]: ";
            cin >> data[i][j];
        }
    }
}

/**
 * @brief Метод вывода матрицы в консоль.
 * Выводит элементы по строкам и столбцам в виде матрицы.
 */
template<typename T>
void TMatrix<T>::output() const {
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            cout << data[i][j] << " ";
        }
        cout << endl;
    }
}

/**
 * @brief Подсчёт элементов > 0 и < среднего по строке.
 * Для каждой строки вычисляется среднее арифметическое.
 * Затем подсчитывается количество элементов в строке, которые удовлетворяют условиям.
 *
 * @return Общее количество таких элементов во всей матрице.
 */
template<typename T>
int TMatrix<T>::countPositiveBelowRowMean() const {
    int count = 0; // Количество нужных элементов
    for (int i = 0; i < rows; ++i) {
        T sum = T(0); // Сумма элементов строки
        for (int j = 0; j < cols; ++j) {
            sum = sum + data[i][j];
        }
        T mean = sum / static_cast<T>(cols); // Среднее арифметическое по строке

        for (int j = 0; j < cols; ++j) {
            if (data[i][j] > T(0) && data[i][j] < mean) {
                count++;
            }
        }
    }
    return count;
}

/**
 * @brief Расчёт суммы отрицательных элементов.
 * Перебирает все элементы матрицы и суммирует только те, что < T(0).
 *
 * @return Сумму отрицательных элементов нужного типа данных.
 */
template<typename T>
T TMatrix<T>::sumOfNegativeElements() const {
    T sum = T(0); // Сумма подходящих элементов
    /// Проходимся по каждому элементу и прибавляем значение к сумме, если меньше нуля
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
```

```
221         for (int j = 0; j < cols; ++j) {
222             if (data[i][j] < T(0)) {
223                 sum = sum + data[i][j];
224             }
225         }
226     }
227     return sum;
228 }
229
230 /**
231  * @brief Метод генерации случайных значений.
232  * Заполняет матрицу случайными значениями в диапазоне `[min_val, max_val]`.
233  * Использует линейную интерполяцию: `val = min_val + t * (max_val - min_val)`,
234  * где `t` — случайное число от 0 до 1.
235  *
236  * @param min_val Минимальное значение для генерации.
237  * @param max_val Максимальное значение для генерации.
238  */
239 template<typename T>
240 void TMatrix<T>::GenerateRandomValues(T min_val, T max_val) {
241     // Используем статическую переменную для инициализации генератора один раз
242     static random_device rd;
243     static mt19937 gen(rd());
244     static uniform_real_distribution<double> dis(0.0, 1.0); // Генерим от 0 до 1
245
246     for (int i = 0; i < rows; ++i) {
247         for (int j = 0; j < cols; ++j) {
248             double t = dis(gen);
249             T range = max_val - min_val;
250             T val = min_val + static_cast<T>(t) * range;
251             data[i][j] = val;
252         }
253     }
254 }
255
256 /**
257  * @brief Модификация элемента матрицы вручную.
258  * Запрашивает новое значение для элемента по указанным индексам.
259  *
260  * @param row Индекс строки.
261  * @param col Индекс столбца.
262  */
263 template<typename T>
264 void TMatrix<T>::modifyElement(int row, int col) {
265     if (row >= 0 && row < rows && col >= 0 && col < cols) {
266         cout << "Enter new value for [" << row << "][" << col << "]: ";
267         cin >> data[row][col];
268     } else {
269         cout << "ERROR: Index out of bounds!" << endl;
270     }
271 }
272
```

```cpp
/**
 * @brief Установка случайного значения для одного элемента.
 * Аналогично GenerateRandomValues, но только для одного элемента.
 *
 * @param row Индекс строки.
 * @param col Индекс столбца.
 * @param min_val Минимальное значение.
 * @param max_val Максимальное значение.
 */
template<typename T>
void TMatrix<T>::SetRandomValue(int row, int col, T min_val, T max_val) {
    if (row < 0 || row >= rows || col < 0 || col >= cols) {
        cout << "ERROR: Index out of bounds!" << endl;
        return;
    }

    // Используем статическую переменную для инициализации генератора один раз
    static random_device rd;
    static mt19937 gen(rd());
    static uniform_real_distribution<double> dis(0.0, 1.0); // Генерим от 0 до 1

    double t = dis(gen);
    T range = max_val - min_val;
    T val = min_val + static_cast<T>(t) * range;
    data[row][col] = val;
}

/**
 * @brief Загрузка матрицы из файла.
 * Формат файла: первые два числа — количество строк и столбцов.
 * Затем идут элементы по строкам.
 *
 * @param filename Имя файла для загрузки.
 */
template<typename T>
void TMatrix<T>::loadFromFile(const string& filename) {
    ifstream file(filename);
    if (!file.is_open()) {
        cout << "ERROR: Cannot open file " << filename << endl;
        return;
    }

    file >> rows >> cols;
    data.resize(rows, vector<T>(cols));

    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            file >> data[i][j];
        }
    }
    file.close();
    cout << "Data loaded from " << filename << endl;
}
```

```cpp
/**
 * @brief Сохранение матрицы в файл.
 * Формат файла: первые два числа — количество строк и столбцов.
 * Затем идут элементы по строкам.
 *
 * @param filename Имя файла для сохранения.
 */
template<typename T>
void TMatrix<T>::saveToFile(const string& filename) const {
    ofstream file(filename);
    if (!file.is_open()) {
        cout << "ERROR: Cannot open file " << filename << endl;
        return;
    }

    file << rows << " " << cols << endl;
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            file << data[i][j] << " ";
        }
        file << endl;
    }
    file.close();
    cout << "Data saved to " << filename << endl;
}

#endif
```

Файл **Fraction.h**:

```cpp
#ifndef FRACTION_H
#define FRACTION_H

#include <iostream>
#include <cmath>
using namespace std;

/**
 * @brief Дробное число, состоящее из числителя и знаменателя.
 */
class Fraction {
private:
    // Числитель
    int num;
    // Знаменатель
    int den;

    /**
     * @brief Вспомогательная функция для вычисления наибольшего общего делителя (НОД).
     * Используется для сокращения дроби.
     *
     * @param a Первое число.
     * @param b Второе число.
     * @return Наибольший общий делитель чисел a и b.
     */
    int gcd(int a, int b) const {
        a = abs(a); // Берем абсолютное значение, чтобы НОД был положительным
        b = abs(b);
        while (b) { // Алгоритм Евклида
            int temp = b;
            b = a % b;
            a = temp;
        }
        return a;
    }

    /**
     * @brief Вспомогательная функция для нормализации дроби.
     * Приводит дробь к каноническому виду: знаменатель положительный, дробь сокращена.
     */
    void normalize() {
        if (den < 0) { // Если знаменатель отрицательный, переносим знак на числитель
            num = -num;
            den = -den;
        }
        int g = gcd(num, den); // Вычисляем НОД
        if (g != 0) { // Если НОД не равен 0, сокращаем дробь
            num /= g;
            den /= g;
        }
    }

public:
    /**
     * @brief Конструктор из двух целых чисел.
     * Создаёт дробь num/den. Выполняет нормализацию.
     *
     * @param n Числитель. По умолчанию 0.
     * @param d Знаменатель. По умолчанию 1.
     */
    Fraction(int n = 0, int d = 1) : num(n), den(d) {
        if (den == 0) { // Защита от деления на ноль
```

```
        if (den == 0) { // Защита от деления на ноль
            cout << "ERROR: Denominator cannot be zero!" << endl;
            den = 1; // Устанавливаем в 1, чтобы избежать сбоя
        }
        normalize(); // Приводим к каноническому виду
    }

    /**
     * @brief Конструктор из double.
     * Преобразует вещественное число в дробь с ограниченной точностью.
     * Например, 0.1 -> 1/10.
     *
     * @param d Вещественное число для преобразования.
     */
    Fraction(double d) {
        const int precision = 100; // Максимальный знаменатель ~100 (2 знака после запятой)
        double scaled_val = d * precision; // Масштабируем до целого
        int n = static_cast<int>(round(scaled_val)); // Округляем до ближайшего целого
        int g = gcd(n, precision); // НОД для сокращения
        num = n / g;
        den = precision / g;
        normalize(); // Финальная нормализация
    }

    // Возвращает числитель
    int getNum() const { return num; }
    // Возвращает знаменатель
    int getDen() const { return den; }

    /**
     * @brief Оператор "меньше".
     * Сравнивает две дроби, приводя к общему знаменателю.
     *
     * @param other Вторая дробь для сравнения.
     * @return true, если текущая дробь меньше, иначе false.
     */
    bool operator<(const Fraction& other) const {
        return (num * other.den) < (other.num * den); // Приведение к общему знаменателю перед сравнением
    }

    /**
     * @brief Оператор "больше".
     *
     * @param other Вторая дробь для сравнения.
     * @return true, если текущая дробь больше, иначе false.
     */
    bool operator>(const Fraction& other) const {
        return (num * other.den) > (other.num * den);
    }

    /**
     * @brief Оператор "равно".
     *
     * @param other Вторая дробь для сравнения.
     * @return true, если дроби равны (с учётом сокращения), иначе false.
     */
    bool operator==(const Fraction& other) const {
        return (num == other.num && den == other.den);
    }
```

```
120     }
121
122     /**
123      * @brief Оператор "не равно".
124      *
125      * @param other Вторая дробь для сравнения.
126      * @return true, если дроби не равны, иначе false.
127      */
128     bool operator!=(const Fraction& other) const {
129         return !(*this == other); // Использует operator==
130     }
131
132     /**
133      * @brief Оператор "меньше или равно".
134      *
135      * @param other Вторая дробь для сравнения.
136      * @return true, если текущая дробь <= other, иначе false.
137      */
138     bool operator<=(const Fraction& other) const {
139         return (*this < other) || (*this == other);
140     }
141
142     /**
143      * @brief Оператор "больше или равно".
144      *
145      * @param other Вторая дробь для сравнения.
146      * @return true, если текущая дробь >= other, иначе false.
147      */
148     bool operator>=(const Fraction& other) const {
149         return (*this > other) || (*this == other);
150     }
151
152     // Операторы арифметики
153     /**
154      * @brief Оператор сложения.
155      * Складывает две дроби: a/b + c/d = (a*d + c*b) / (b*d).
156      *
157      * @param other Вторая дробь.
158      * @return Новую дробь, равную сумме.
159      */
160     Fraction operator+(const Fraction& other) const {
161         return Fraction(num * other.den + other.num * den, den * other.den);
162     }
163
164     /**
165      * @brief Оператор вычитания.
166      * Вычитает: a/b - c/d = (a*d - c*b) / (b*d).
167      *
168      * @param other Вторая дробь.
169      * @return Новую дробь, равную разности.
170      */
171     Fraction operator-(const Fraction& other) const {
172         return Fraction(num * other.den - other.num * den, den * other.den);
173     }
```

```
175     /**
176      * @brief Оператор умножения.
177      * Умножает: a/b * c/d = (a*c) / (b*d).
178      *
179      * @param other Вторая дробь.
180      * @return Новую дробь, равную произведению.
181      */
182     Fraction operator*(const Fraction& other) const {
183         return Fraction(num * other.num, den * other.den);
184     }
185 
186     /**
187      * @brief Оператор деления.
188      * Делит: a/b / c/d = (a*d) / (b*c).
189      * Проверяет деление на ноль.
190      *
191      * @param other Дробь-делитель.
192      * @return Новую дробь, равную частному.
193      */
194     Fraction operator/(const Fraction& other) const {
195         if (other.num == 0) { // Проверка деления на ноль
196             cout << "ERROR: Division by zero!" << endl;
197             return Fraction(0, 1); // Возвращаем ноль
198         }
199         return Fraction(num * other.den, den * other.num);
200     }
201 
202     // Проверяет, положительна ли дробь
203     bool isPositive() const { return num > 0; }
204     // Проверяет, отрицательна ли дробь
205     bool isNegative() const { return num < 0; }
206 
207     /**
208      * @brief Преобразование дроби в double.
209      * Может использоваться для приближённых вычислений или генерации случайных значений.
210      *
211      * @return Значение дроби как double.
212      */
213     double toDouble() const {
214         return static_cast<double>(num) / den;
215     }
216 
217     /**
218      * @brief Оператор ввода из потока.
219      * Читает дробь в формате "num/den" или "num" (если знаменатель 1).
220      *
221      * @param is Входной поток (например, cin или ifstream).
222      * @param f Объект дроби для заполнения.
223      * @return Ссылку на входной поток.
224      */
225     friend istream& operator>>(istream& is, Fraction& f) {
226         string s;
227         is >> s; // Читаем всю строку до пробела
228 
229         size_t pos = s.find('/'); // Ищем символ '/'
230         if (pos == string::npos) { // Если '/' нет — это целое число
231             f.num = stoi(s);
232             f.den = 1;
233         } else { // Если '/' есть — разбиваем строку
234             string num_str = s.substr(0, pos); // До '/'
```

```
234                 string num_str = s.substr(0, pos); // До '/'
235                 string den_str = s.substr(pos + 1); // После '/'
236                 f.num = stoi(num_str);
237                 f.den = stoi(den_str);
238                 if (f.den == 0) { // Проверка на деление на ноль
239                     cout << "ERROR: Denominator cannot be zero! Setting to 1." << endl;
240                     f.den = 1;
241                 }
242             }
243             f.normalize(); // Нормализуем дробь после ввода
244             return is;
245         }
246 
247         /**
248          * @brief Оператор вывода в поток.
249          * Выводит дробь в формате "num/den", если знаменатель != 1, иначе только "num".
250          *
251          * @param os Выходной поток (например, cout или ofstream).
252          * @param f Объект дроби для вывода.
253          * @return Ссылку на выходной поток.
254          */
255         friend ostream& operator<<(ostream& os, const Fraction& f) {
256             os << f.num; // Выводим числитель
257             if (f.den != 1) os << "/" << f.den; // Если знаменатель != 1, выводим "/"
258             return os;
259         }
260     };
261 
262     #endif
```

Файл **main.cpp**:

```cpp
#include "TMatrix.h"
#include "Fraction.h"
#include <iostream>
using namespace std;

/// @brief Шаблонная функция, реализующая меню для конкретного типа данных
/// @tparam T тип данных элементов матрицы
/// @param datatype_name название типа данных
template<typename T>
void runMatrixMenu(string datatype_name) {
    TMatrix<T> matrix;
    bool running = true;

    while (running) {
        cout << "\n====================================" << endl;
        cout << "      TMatrix MENU (Type: " << datatype_name << ")" << endl;
        cout << "====================================" << endl;
        cout << "\n--- OPERATIONS ON MATRIX ---" << endl;
        cout << "1. Create new matrix from input" << endl;
        cout << "2. Display current matrix" << endl;
        cout << "3. Count positive elements below row mean" << endl;
        cout << "4. Sum of negative elements" << endl;
        cout << "5. Load matrix from file" << endl;
        cout << "6. Save current matrix to file" << endl;
        cout << "7. Modify an element (console input)" << endl;
        cout << "8. Set random value for an element" << endl;
        cout << "9. Generate random values for the whole matrix" << endl;
        cout << "10. Demonstration of copy constructor" << endl;
        cout << "0. Go back to type selection" << endl;
        cout << "Choose an action: ";

        int choice;
        cin >> choice;

        switch (choice) {
            case 1: { // Ввод матрицы
                matrix.input();
                cout << "Matrix created." << endl;
                break;
            }
            case 2: { // Вывод матрицы
                cout << "\n--- Current Matrix ---" << endl;
                cout << matrix << endl;
                break;
            }
            case 3: { // Счёт положительных элементов > среднего значения
                cout << "\nCount of positive elements below row mean: " << matrix.countPositiveBelowRowMean() << endl;
                break;
            }
            case 4: { // Сумма отрицательных элементов
                auto sum = matrix.sumOfNegativeElements();
                cout << "\nSum of negative elements: " << sum << endl;
                break;
            }
            case 5: { // Загрузка матрицы из файла
                string filename;
                cout << "Enter filename to load from: ";
                cin >> filename;
                matrix.loadFromFile(filename);
                break;
            }
            case 6: { // Выгрузка матрицы в файл
```

```
            case 6: { // Выгрузка матрицы в файл
                string filename;
                cout << "Enter filename to save to: ";
                cin >> filename;
                matrix.saveToFile(filename);
                break;
            }
            case 7: { // Модификация элемента
                int r, c;
                cout << "Enter row index: ";
                cin >> r;
                cout << "Enter column index: ";
                cin >> c;
                matrix.modifyElement(r, c);
                break;
            }
            case 8: { // Задать случайное значение элементу
                int r, c;
                cout << "Enter row index: ";
                cin >> r;
                cout << "Enter column index: ";
                cin >> c;
                T min_val, max_val;
                cout << "Enter min value for random: ";
                cin >> min_val;
                cout << "Enter max value for random: ";
                cin >> max_val;
                matrix.SetRandomValue(r, c, min_val, max_val);
                break;
            }
            case 9: { // Сгенерировать случайные значения всем элементам матрицы
                T min_val, max_val;
                cout << "Enter min value for random: ";
                cin >> min_val;
                cout << "Enter max value for random: ";
                cin >> max_val;
                matrix.GenerateRandomValues(min_val, max_val);
                cout << "Random values generated." << endl;
                break;
            }
            case 10: { // Демонстрация конструктора копирования
                cout << "--- Demonstration of copy constructor ---" << endl;
                cout << "Current Matrix" << endl;
                cout << matrix << endl;
                TMatrix<T> temp_copy = matrix;
                cout << "Copy Of Current Matrix" << endl;
                cout << temp_copy << endl;
                break;
            }
            case 0: { // Возврат к выбору типа
                running = false;
                cout << "Returning to type selection..." << endl;
                break;
            }
            default: { // Неверный ввод
                cout << "Invalid choice. Please try again." << endl;
            }
        }
    }
```

```
123 int main() {
124     bool running = true;
125 
126     while (running) {
127         cout << "\n=====================================" << endl;
128         cout << "            MAIN MENU" << endl;
129         cout << "=====================================" << endl;
130         cout << "\n--- SELECT MATRIX TYPE ---" << endl;
131         cout << "1. Work with Float Matrix" << endl;
132         cout << "2. Work with Fraction Matrix" << endl;
133         cout << "0. Exit" << endl;
134         cout << "Choose an action: ";
135 
136         int choice;
137         cin >> choice;
138 
139         switch (choice) {
140             case 1: { // Переключение на матрицу вещественных чисел
141                 cout << "Switched to Float Matrix." << endl;
142                 runMatrixMenu<float>("Float"); // Вызываем шаблонную функцию с типом float
143                 break;
144             }
145             case 2: { // Переключение на матрицу дробей
146                 cout << "Switched to Fraction Matrix." << endl;
147                 runMatrixMenu<Fraction>("Fraction"); // Вызываем шаблонную функцию с типом Fraction
148                 break;
149             }
150             case 0: { // Выход из программы
151                 running = false;
152                 cout << "Exiting..." << endl;
153                 break;
154             }
155             default: { // Неверный ввод
156                 cout << "Invalid choice. Please try again." << endl;
157             }
158         }
159     }
160 
161     return 0;
162 }
```

Создание матрицы с помощью ручного ввода и её вывод в консоль:

```
=====================================
          MAIN MENU
=====================================

--- SELECT MATRIX TYPE ---
1. Work with Float Matrix
2. Work with Fraction Matrix
0. Exit
Choose an action: 1
Switched to Float Matrix.

=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 1
Enter number of rows: 2
Enter number of columns: 2
Enter element [0][0]: 1
Enter element [0][1]: 4.0
Enter element [1][0]: -2.3
Enter element [1][1]: -16.7
Matrix created.
```

```
--- Current Matrix ---
1 4
-2.3 -16.7
```

Счёт положительных элементов ниже среднего арифметического значения строки:

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 3

Count of positive elements below row mean: 1
```

Сумма отрицательных элементов матрицы:

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 4

Sum of negative elements: -19
```

Изменение элемента матрицы (ручной ввод):

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 7
Enter row index: 0
Enter column index: 1
Enter new value for [0][1]: 20.5
```

```
--- Current Matrix ---
1 20.5
-2.3 -16.7
```

Изменение элемента матрицы (рандомизация):

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 8
Enter row index: 0
Enter column index: 0
Enter min value for random: 1
Enter max value for random: 5
```

```
--- Current Matrix ---
4.32172 20.5
-2.3 -16.7
```

Рандомизация всех элементов матрицы:

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 9
Enter min value for random: 5.0
Enter max value for random: 10.0
Random values generated.
```

```
--- Current Matrix ---
6.43958 7.4125
6.41653 7.13014
```

Копирование матрицы с помощью конструктора копирования:

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 10
--- Demonstration of copy constructor ---
Current Matrix
6.43958 7.4125
6.41653 7.13014

Copy Of Current Matrix
6.43958 7.4125
6.41653 7.13014
```

Запись данных массива в файл:

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 6
Enter filename to save to: text1.txt
Data saved to text1.txt
```

```
text1.txt U ×
C_codespace > 2 kurs > 3-rd senester
1   2 2
2   6.43958 7.4125
3   6.41653 7.13014
4
```

Переключение на матрицу с дробными числами:

```
=====================================
      TMatrix MENU (Type: Float)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 0
Returning to type selection...

=====================================
           MAIN MENU
=====================================

--- SELECT MATRIX TYPE ---
1. Work with Float Matrix
2. Work with Fraction Matrix
0. Exit
Choose an action: 2
Switched to Fraction Matrix.
```

Загрузка данных из файла:

```
=====================================
      TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 5
Enter filename to load from: input_fraction
Data loaded from input_fraction
```

```
--- Current Matrix ---
1/2 1/3
-1/4 -1/5
```

Счёт положительных элементов ниже среднего арифметического значения строки и сумма отрицательных элементов матрицы:

```
Count of positive elements below row mean: 1

Sum of negative elements: -9/20
```

Изменения элемента матрицы при вводе с консоли и рандомизация элемента матрицы:

```
=====================================
      TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 7
Enter row index: 0
Enter column index: 0
Enter new value for [0][0]: 2/10

=====================================
      TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 8
Enter row index: 0
Enter column index: 1
Enter min value for random: 1/10
Enter max value for random: 1/2

--- Current Matrix ---
1/5 89/250
-1/4 -1/5
```

Рандомизация всех элементов матрицы:

```
=====================================
      TMatrix MENU (Type: Fraction)
=====================================

--- OPERATIONS ON MATRIX ---
1. Create new matrix from input
2. Display current matrix
3. Count positive elements below row mean
4. Sum of negative elements
5. Load matrix from file
6. Save current matrix to file
7. Modify an element (console input)
8. Set random value for an element
9. Generate random values for the whole matrix
10. Demonstration of copy constructor
0. Go back to type selection
Choose an action: 9
Enter min value for random: 1/20
Enter max value for random: 1/5
Random values generated.
```

```
--- Current Matrix ---
41/250 137/1000
73/500 59/400
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Какой принцип ООП реализуется с помощью шаблонных функций и классов?**
   Обобщение (Generics) — возможность создания кода, работающего с разными типами данных, без дублирования.
2. **Когда целесообразно использовать шаблоны функций?**
   Когда нужно выполнить одинаковую логику над данными разных типов.
3. **Каков общий вид объявления шаблона функции?**
   template<typename T> тип_возврата имя_функции(параметры)
4. **Каков синтаксис объявления шаблонного класса?**
   template<typename T> class ИмяКласса { ... };
5. **В чём особенности определения шаблонного класса?**
   Все методы и операторы внутри класса также становятся шаблонами.
6. **В чём особенности определения метода шаблонного класса вне класса?**
   Перед определением метода нужно снова указать template<typename T> (или template<class T>). Сигнатура метода включает имя шаблонного класса с параметром:
   template<typename T> void TClassName<T>::methodName(...) { ... }.
7. **Что такое специализация шаблона?**
   Это частичная или полная замена параметра шаблона конкретным типом, чтобы предоставить особую реализацию для этого типа.
8. **Как объявить объект шаблонного класса?**
   ИмяКласса<конкретный_тип> имя_объекта; (например, TMatrix<int> intMatrix;).
9. **Что такое инстанцирование?**
   Процесс создания конкретной версии шаблона (функции или класса) для определенного типа компилятором.
10. **Чем в строке template<class T> может быть T?**
    d) T может быть любым типом – как встроенным в язык C++, так и определённым в коде программы.

ВЫВОДЫ

Я получил практические навыки разработки и отладки программ с использованием шаблонов функций и классов на языке C++.